

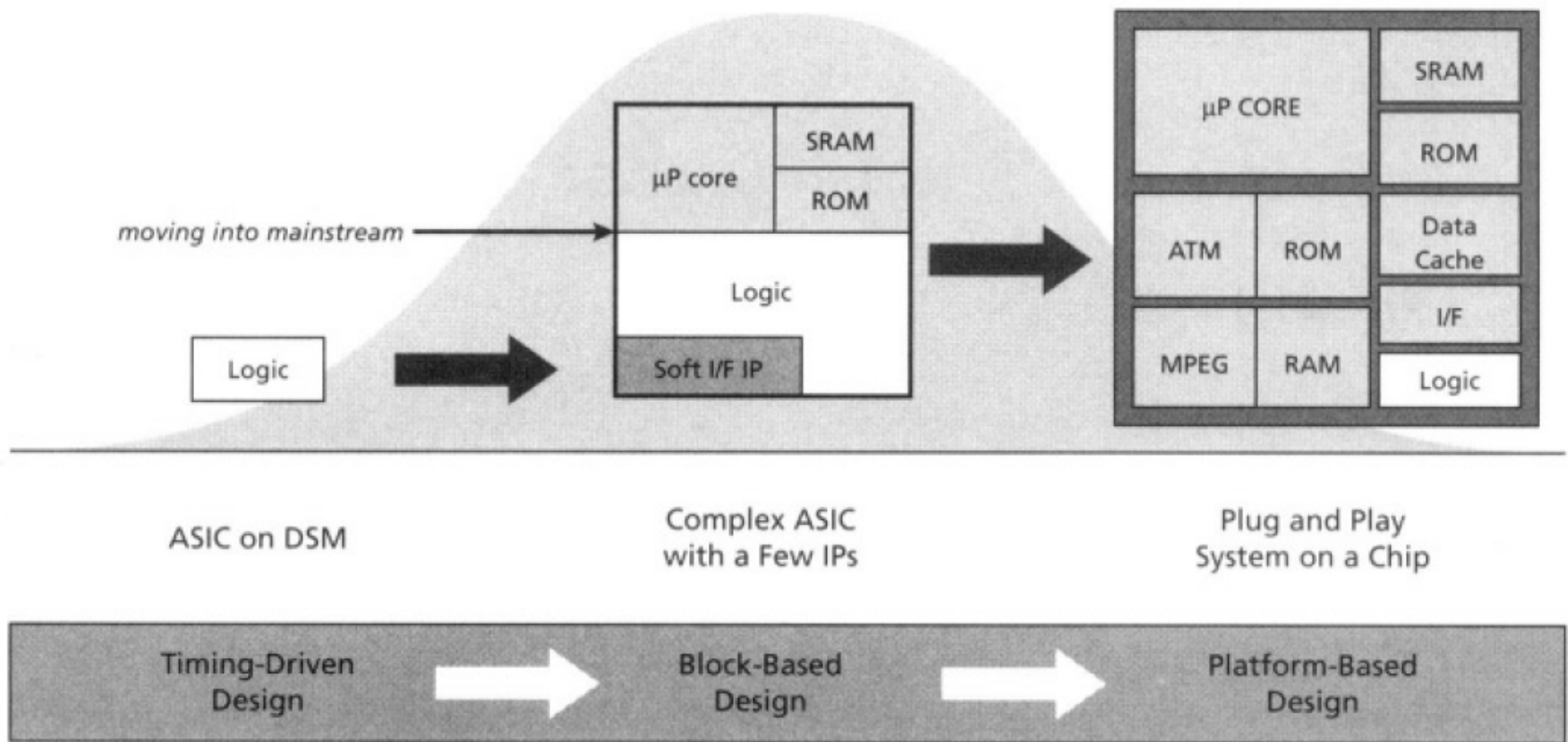
SOC Design Methodologies

SOC

- An SOC is a system on an IC that integrates software and hardware Intellectual Property (IP) using more than one *design methodology* for the purpose of defining the functionality and behavior of the proposed system.

Evolution of Design Methodology

- **Back in 1980s**, the IC design processes were not matured enough and could only support large feature size, resulting in large chip area even for simpler designs. Designs were made using *Area Driven Design Methodology* with a view point of keeping the chip area minimum.
- **In 90s**, with scaling and IC design process evolution, area of the designs starts shrinking and design goal shifted to performance and designs were made using *Timing Driven Design Methodology* with a view point of keeping the Timing/Delay minimum.
- **In 2000**, with rise in market competition, with the advent of EDA tools, ASIC vendors tried to add more functionality in the chip and with lesser time to market. So *Block Based Design Methodology* came into existence.
- **To further decrease the TTM** for the first design and derivative products, vendors started reusing the existing designs and standardization of design were performed. This type of design method is known as *Platform Based Design*.



SOC Design Methodologies

- The primary design methods used today can be divided into three segments:
 - Timing Driven Design (TDD)
 - Block Based Design (BBD)
 - Platform Based Design (PBD)
- These segments vary depending on the
 - Linchpin technologies used,
 - Design capacity,
 - Level of and investment in design reuse.

Linchpin Technology

- Linchpin means something Indispensable.
- Linchpin Technology is the procedures/rules vital to a design process i.e. *“a particular design methodology can be successful by use of certain procedures, these procedures make the Linchpin Technology for that particular design methodology”*.
- TDD, BBD, PBD have their own linchpin technologies.

Timing Driven Design

- TDD is the best approach for designing moderately sized and complex ASICs, consisting primarily of new logic (little if any reuse) on DSM processes, without a significant utilization of hierarchical design.
- The design methodology prior to TDD is area-driven design (ADD).
In ADD, logic minimization was the key objective.
- Design teams using this methodology tend to be small and homogeneous.
- When designer encounter problems in meeting performance or power constraints it requires shifting to a TDD methodology.

TDD Linchpin Technologies

- TDD relies upon the following linchpin technologies:

- **Interactive floor-planning tools:**

These give accurate delay and area estimates earlier in the design process, thereby addressing the timing and area convergence problem between synthesis and place and route.

- **Static-timing analysis tools:**

These enable a designer to identify timing problems quickly and perform timing optimization across the entire ASIC.

The designer can perform most functional verification at RTL with simpler timing views, reduce the amount of slower timing-accurate gate-level simulations, and rely upon static timing analysis to catch any potential timing-related errors, thereby improving productivity significantly.

TDD Linchpin Technologies

- **Using compilers to move design to higher abstractions with timing predictability**

A behavioral synthesis tool can be linked to a data path compiler, providing an operational vehicle for planning and implementing data path-dominated designs rapidly.

This moves the critical decision trade-offs into the behavioral level, while backing it up with a high-performance path to an efficient data path layout.

Applied appropriately, it can radically improve a design's overall performance.

It also introduces layout optimization at the system level, which is needed in block- and platform-based designs.

TDD Linchpin Technologies

TDD Benefits

- Netlist handoff is well understood; floor plan centric method limits iterations due to timing discrepancies for most designs.
- Flat-timing analysis/delay calculation is more accurate and better supported by most of today's tools.
- Accommodates RAM/ROM blocks as cells. Compilers can be used to improve designer productivity.
- Test generation can be uniform and automatic.
- Can handle small, soft VC blocks as design sources.

TDD Challenges

- High-performance, high-density DSM products still require iterative loops from place and route back into timing analysis/synthesis.
- Design changes resulting from RTL simulation are very difficult to incorporate the physical design, impacting TTM.
- Flat methodology begins to fail as complexity increases and gate counts rise above 150K.
- Synthesis tools limit block sizes; wire-load models are insufficient for high-performance or low-power designs.
- Turnaround time through ASIC physical design is long and subject to non-convergence.

Block-Based Design

- Increasing design complexity and an increasing opportunistic reuse of system-level functions are reasons to move beyond TDD methodology.
- **BBD is the behaviorally modeled design at the system level**, where hardware/software trade-offs and functional hardware/software co-verification using software simulation and/or hardware emulation is performed.
- The new design components are then partitioned and mapped onto specified functional RTL blocks, which are then designed to budget timing, power, and area constraints.

Block-Based Design (Cont...)

- This is in contrast to the TDD approach, where the RTL model is captured.
- BBD needs an effective block-level floor planner that can quickly estimate RTL block sizes. Creating viable budgets for all the blocks and their interconnect is essential to achieving convergence.
- This convergence can be significantly improved through the use of synthesis tools.

Block-Based Design (Cont...)

- Symptoms to look for in determining whether a BBD methodology is more appropriate include:
 - The design team is becoming more application-specific, and subsystems, such as embedded processing, digital data compression, and error correction, are required.
 - Multiple design teams are formed to work on specific parts of the design.
 - ASIC engineers are having difficulty developing realistic and comprehensive test benches.
 - Interface timing errors between subsystems are increasing dramatically.
 - The design team is looking for VCs outside of their group to help accelerate product development.

BBD Linchpin Technologies

- BBD relies upon the following linchpin technologies:
 - **Application-specific, high-level system algorithmic analysis tools:**

These tools provide productivity in modeling system algorithms and system operating environment.

They can be linked to hardware description language (HDL) verification tools (Verilog, VHDL).
 - **Integrated synthesis and physical design:**

This technology enables a designer to manage the increased influence of physical design effects during the synthesis process by eliminating the need to iterate between separate synthesis and placement and routing tools to achieve design convergence.

BBD Linchpin Technologies

BBD Benefits	BBD Challenges
Top-down design approach manages complexity for large designs.	Process does not create a set of reusable components.
Import of external processor cores supported.	TTM constrained by design and re-verification of soft RTL blocks. Process encourages design tweaking of blocks.
Complex, parallel block design teams supported.	Limited availability of cycle-accurate and behavioral models. Tracking models against RTL difficult and expensive.
DSM interconnect effects managed through RTL floor planning (including placement and budgets), datapath, and global interconnect.	Flat-routing and flat-delay calculation approach moves potential interconnect problems to late in the design process.
Link between system and RTL simulations minimizes unique testbench development, and provides confidence that realistic and worst-case scenarios are being verified.	Hardware/software verification often requires a high-performance hardware emulation environment, a consequence of the lack of high-speed, high-level models.

Platform-Based Design

- PBD is the next step in the technology evolution. PBD encompasses the cumulative capabilities of the TDD and BBD technologies, plus extensive design reuse and design hierarchy.
- Like BBD, PBD is a hierarchical design methodology that starts at the system level. Where PBD differs from BBD is that it achieves its high productivity through extensive, planned design reuse.
- Productivity is increased by using predictable, pre verified blocks that have standardized interfaces.
- The better planned the design reuse, the less changes are made to the functional blocks. PBD can decrease the overall TTM for first products, and expand the opportunities and speed of delivering derivative products.

Platform-Based Design (Cont...)

- PBD methodology separates design into two areas of focus:

- **Block authoring/creation:**

- The block is created so that it interfaces easily with multiple target designs.

- Many different design teams, both internal and external to the company, can do block authoring, as long as they are all using the same interface specifications.

- **System-chip integration:**

- System-chip integration focuses on designing and verifying the system architecture and the interfaces between the blocks.

- The deliverables from the block author to the system integrator are standardized.

PBD Linchpin Technologies

- PBD is enabled by the following linchpin technologies:
 - High-level, system-level algorithmic and architectural design tools and hardware/software co-design technologies
 - Physical layout tools focused on bus planning and block integration
 - VC-authoring functional verification tools

PBD Linchpin Technologies

PBD Benefits	PBD Challenges
Planned design reuse yields very high productivity.	Planned reuse requires significant up-front design planning and accurate future product plans.
Designs can be composed of diverse, specialized functions from multiple sources.	Significant software portions require extensive hardware/software co-verification.
Hierarchical routing and timing reduce design focus completely.	Embedded AMS design content creates noise and test issues.
Interface-based design promotes higher abstraction design and implementation.	Platform migration to new process technology requires re-characterization of VCs (hard and soft) and platform architecture.
Multiple reuse for blocks allows for development cost amortization and more optimal block design.	Requires organizational change to support separated block authoring and system-chip integration.

Difference between TDD, BBD, PBD

Thanks